

PRACTICE Qs VIDEO: LC-1365 HOW MANY NUMBERS ARE SMALLER THAN THE CURRENT NUMBER

GIVEN THE ARRAY "NUMS" FOR EACH  $\text{nums}[i]$  FIND OUT HOW MANY NUMBERS IN THE ARRAY ARE SMALLER THAN IT. THAT IS, FOR EACH  $\text{nums}[i]$  YOU HAVE TO COUNT THE NUMBER OF VALID  $j$ 's SUCH THAT  $j \neq i$   $\text{nums}[j] < \text{nums}[i]$

CONSTRAINTS:

- RETURN THE COUNT OF NUMBERS FIRST
- THEN RETURN AN ARRAY OF THOSE NUMBERS

EX:  $\text{nums} = [8, 1, 2]$

OUTPUT: FOR  $\text{nums}[0] = 8$  THERE ARE 2 SMALLER NUMS (1, 2)

FOR  $\text{nums}[1] = 1$  THERE ARE 0 SMALLER NUMS

FOR  $\text{nums}[2] = 2$  THERE ARE 1 SMALLER NUMS (1)

SO WE HAVE TO GO THROUGH EACH ELEMENT ONCE

↳ BUT FOR EACH IT, WE COULD USE A NESTED LOOP AND WE JUST NEED TO ITERATE AND

CHECK ALL NUMBERS OTHER THAN THE ONE AT  $i$ .

(THIS IS  $O(N^2)$  TIME COMPLEXITY, HOW DO WE AVOID THIS?)

MAYBE SORTING THE LIST WOULD HELP. SORTING IS AN  $O(N \log N)$  OPERATION  
↳  $[8, 1, 2] \rightarrow [1, 2, 8]$  NOW WE KNOW THE # SMALLER BASED ON THE INDEX

0 1 2  
1 HAS 0 SMALLER, 2 HAS 1 SMALLER. # NUMS  $\times$  SMALLER  
LARGE  $x$  IS ITS SORTED INDEX

THEN WE WOULD STILL NEED TO LOOP FROM  $0 \rightarrow x$  TO PRINT THOSE SMALLER NUMS

CAN THIS BE AVOIDED? YES, IN A SORTED LIST IT CAN BECAUSE THE PATTERN

ACCUMULATES

SO AT  $x$  WE PRINT. SO WE CAN LEVERAGE THIS PATTERN TO ITERATE THROUGH THE ARRAY

THEN AT  $x+1$  WE PRINT 1. ONCE AND HAVE THE PREVIOUS RESULT ACCUMULATE. THIS WON'T WORK IF 2 OF THE SAME NUMBER EXISTS

1, 2

TOTAL TIME COMPLEXITY:  $O(N \log N) + O(N)$

### MY ALGORITHM: (BASED ON 1 & 2)

1. CREATE A SORTED COPY OF THE ORIGINAL ARRAY  $O(N \log N)$

2. CREATE AN EMPTY MAP

3. CREATE AN EMPTY STRING FOR OUR ACCUMULATED RESULT

4. ITERATE THROUGH THE SORTED ARRAY:

↳  $\text{ADD}[\text{nums}[i]:i]$  ONLY IF IT DOES NOT ALREADY EXIST

THIS ENSURES THAT WE DO NOT MISREPRESENT DUPLICATES

\*ALSO, ADD  $\text{STRUCT } \{i, \text{RES}\}$   $O(N)$

5. NOW ITERATE THROUGH THE ORIGINAL ARRAY

\*THIS PART IS EASY SINCE THE ANSWER IS ALREADY IN OUR MAP

↳ JUST PRINT A FORMATTED RESULT GIVEN:

CUR INDEX, NUM, # LESS (FROM MAP), RESULT STR (FROM MAP)

### VIDEO SOLUTION:

\*HINT: ITS SIMILAR TO 2 SUM • HINT: SORTING HELPS

②

• SORTING IT: MAYBE WE CAN LEVERAGE A MAP TO MAKE MY OLD SOLUTION WORK

$[1, 2, 2, 8]$  FOR EXAMPLE:

1. WE HAVE  $\text{nums}[0] = 1$  SO STORE IN THE MAP  $\{1:0\}$  (NORMAL CASE)

2. NEXT  $\text{nums}[1] = 2$ . STORE IN THE MAP  $\{2:1\}$

BUT NOW  $\text{nums}[2] = 2$  ALSO. SO WE CAN INSTEAD OF OVERWRITING WE INTENTIONALLY DO NOT OVERWRITE BECAUSE THAT WOULD VIOLATE OUR PATTERN.

↳ NOW THIS MEANS WE CAN ALSO ACCUMULATE A STRING OF THOSE SMALLER NUMBERS, WE CAN ALSO STORE THIS IN THE MAP:

THIS WOULD LOOK LIKE:

STRING  $\text{RES} = \text{"ACCUMULATED"}$

THEN WHEN WE INSERT INTO MAP WE ALSO INSERT THIS STRING

\* THEN WE PRINT RESULT AT EACH ITERATION

SO FOR 2, ONLY

1 GETS INSERTED EVEN THOUGH FOR THE 2<sup>ND</sup> 2 WE ACCUMULATED 1, 2

# PRACTICE Qs VIDEO: LC-1365 HOW MANY NUMBERS ARE SMALLER THAN THE CURRENT NUMBER

## VIDEO SOLUTION:

## • O(N) SOLUTION:

```
count = [0] * 102 CREATES ARRAY OF 102 0's
for num in nums:
    count[num+1] += 1
for i in range(1, 102):
    count[i] += count[i-1]
return [count[num] for num in nums]
```

## HOW DOES THIS WORK?

nums: [1, 2, 2, 8]  
 num = 1: count[2] += 1  
 = 2: count[3] += 1  
 = 2: count[3] += 1  
 = 8: count[9] += 1

COUNT:  
 2 3 ... 9  
 1 2 1

COUNT [i] = COUNT [i-1] + COUNT [i] THIS IS A WAY TO ACCUMULATE THROUGHOUT THE ARRAY

• HOW DOES IT LOOK?

COUNT BEFORE:

[0, 0, 1, 2, 0, 0, 0, 0, 0, 1, ...]  
 0 1 2 3 4 5 6 7 8 9

• WHY DO WE DO THIS? THIS APPROACH IS INTUITIVE AND IT DOESN'T EVEN PRINT WHICH NUMBERS ARE LESS.

COUNT AFTER

[0, 0, 1, 3, 3, 3, 3, 3, 3, 4, ...]  
 0 1 2 3 4 5 6 7 8 9

BUT WHAT IS WORTH LEARNING IS HOW WE AVOIDED SORTING?

nums [1, 2, 2, 8] OUR OUTPUT SHOULD BE

1, (0) 2, (1) 8, (3)

USING THE 'MAPPING' CREATES AN ORDERING W/O NEEDING TO SORT. IT TRADES MORE SPACE COMPLEXITY FOR LESS TIME COMPLEXITY

ACCUMULATING WORKS BECAUSE WE MAPPED THE NUMBERS LESS THAN EACH NUM PRECEPTING THAT NUM

HOW WOULD I PRINT THE SPECIFIC NUMBERS THAT WERE LESS THAN NUM?

IN OUR EXAMPLE FOR nums: 1, 2, 2, 8 WE NEED TO PRINT (0), (1), (1, 2, 2)

THE LAST CONTROL FLOW LOOPS THROUGH THE ACCUMULATED ARRAY (w/ jumps bc we use nums as the index)

USE AN ACCUMULATED RESULT STR. THEN, 1. PRINT THE BUFFER

WE NEED TO REALIZE RESUME 0, THE NUMBER IS STILL PRESENT

2. ADD (x+1) i's TO THE BUFFER (THIS WORKS bc OF THIS)